

C++ における 内包的定義に基づいた柔軟な集合データ構造の 設計と実装

26B blueberry

目次

要旨	2
1 はじめに	3
1.1 前提	3
1.2 本レポートの目的	3
2 <code>std::set</code> による集合の表現	3
2.1 基本的な集合操作	3
2.2 集合族	5
2.3 <code>std::set</code> による集合表現の限界	6
3 関数による集合の表現	7
3.1 集合を関数で表現する	7
3.2 集合演算と論理演算の対応	8
3.3 集合の一致判定	12
4 型の一般化	13
4.1 任意の型への対応	13
4.2 型消去による実装	13
4.3 集合族への拡張	17
5 有限集合との融合	17
5.1 実装	17
6 DSL による集合記法	25

6.1	設計方針	25
6.2	記法の例	25
6.3	実装	25
7	まとめ	32
7.1	本ライブラリの特徴	32
7.2	今後の課題	32
8	アピール文	32

要旨

情報理工学基礎の授業で扱われた集合論の諸概念を C++ 上に実装・検証した結果をまとめた。さらに、実装したデータ構造をブラウザ上で実際に動かして試すことができるよう、独自言語 (DSL) の構文解析器を含む Web デモを作成した過程についても述べている。

まず、C++ の標準ライブラリである `std::set` を用いた一般的な集合の表現について検討を行った。`std::set` や標準アルゴリズムを活用することで、要素の追加・削除や和集合・積集合・差集合といった基本的な有限集合の演算を簡潔に記述できることを確認した。しかし、`std::set` は内部で要素を保持する外延的定義に依存しているため、無限集合や補集合を扱えないという限界がある。また、プログラミング言語の型システムによる制約を受け、異なる型の要素を含む集合や集合族の柔軟な表現が困難である点も指摘している。

これらの課題を克服するため、本レポートでは集合に含まれるかを判定する関数を内部で保持する「内包的定義」によるアプローチを採用した。これにより偶数全体などの無限集合を表現可能とし、和集合・積集合・補集合・差集合を論理演算 (OR, AND, NOT など) と対応付けて演算子オーバーロードで実装することで、直感的な集合演算を実現した。ただし、関数による表現ではプログラムの等価性判定が停止性問題に帰着されるため、数学的に完全な集合の一致判定が行えないという理論的限界についても考察している。

続いて、`std::any` による型消去を活用して任意の型を扱えるよう拡張したうえで、関数による「条件定義」と個別の要素を追加・削除する「有限集合」の機能を融合させたハイブリッドなデータ構造を設計・実装した。最後に、ユーザーが直感的に集合記述を行える簡易 DSL のパーサーを構築し、Web サイト上で動作させる仕組みを実現した。本取り組みを通じて、`std::set` が扱えない無限集合などを扱えるようになり、データ構造・集合の概念両方への理解が深まった。今後は計算量の最適化や構文の拡張が課題である。

1 はじめに

1.1 前提

このレポートは、情報理工学基礎の授業で扱われた概念がプログラミング言語でどのように実装できるかを述べるものである。どのプログラミング言語を選ぶかは自由だが、今回は筆者が書き慣れている C++ で実装することにした。特に言及が無ければ C++23 を使用することを前提とする。

1.2 本レポートの目的

本レポートの目的は、C++ 上で集合論のさまざまな概念を出来るだけ包括的に扱えるような実装方法を考え、それを実際に実装して検証することである。作成したプログラムを何かに活用することを考えているわけではないが、できるだけ合理的で、納得感のある実装を目指す。

しかし、レポートで実装内容を説明できる一方で、読者が実際にその動作を試すことが難しい。せっかくなら実際に動くものを作った方が楽しいので、実装したデータ構造の一部の機能を実際に動かすことができる Web サイトを作成することも目的とする。Web 上で動作させるためには、ユーザーに C++ プログラム記述させるのは望ましくない。そこで、集合論のみを記述できる簡潔な独自言語（以降 DSL(Domain-Specific Language) と呼ぶ）を実装する。ユーザーが記述した DSL を入力として受け取り、C++ 上で DSL を解釈したうえで実行する形で Web サイトを作成する。

また、集合論の概念をプログラムで表現することで、集合論を新たな視点から捉えることができるだろう。その方法について理解を深めることも目的の一つと言える。

2 std::set による集合の表現

この章では C++ の標準ライブラリに含まれている `std::set` を使った集合の表現方法について述べる。

2.1 基本的な集合操作

2.1.1 初期化と代入

以下のようにして整数集合の初期化と代入を行うことができる。

```
1 #include<set>
2 int main(){
3     std::set<int> s = {1,2,3};
4     return 0;
5 }
```

以降は `set` が `include` されていることと `std::set<int>s` が宣言されていることを前提とする。

2.1.2 値の追加、削除

```
1 //値の追加
2 s.insert(4);
3 //s = {1,2,3,4}
4
5 //同じ値を追加しても重複は発生しない
6 s.insert(3);
7 //s = {1,2,3,4}
8
9 //値の削除
10 s.erase(1);
11 //s = {2,3,4}
```

2.1.3 存在判定, 列挙

```
1 std::cout << s.contains(4) << std::endl;
2 //1
3
4 std::cout << s.contains(5) << std::endl;
5 //0
6
7 for(int:x:s){
8     std::cout << x << " ";
9 }
10 std::cout << std::endl;
11 //1 2 3 4
```

2.1.4 和集合、差集合、積集合

標準ライブラリにも和集合、差集合、積集合を求める関数が入っている。自分で実装する事も可能だが、標準ライブラリの関数で済ませられるならばそれを使った方が簡潔に書くことができるだろう。

```
1 #include<set>
2 #include<algorithm>
3 int main(){
4     std::set<int> a = {3,6,9};
5     std::set<int> b = {2,4,6};
6
7     //和集合
8     std::set<int> union_set;
```

```

9      std::set_union(a.begin(),a.end(),
10                    b.begin(),b.end(),
11                    std::inserter(union_set,union_set.end()));
12      //union_set = {2,3,4,6,9}
13
14      //差集合
15      std::set<int> diff_set;
16      std::set_difference(a.begin(),a.end(),
17                          b.begin(),b.end(),
18                          std::inserter(diff_set,diff_set.end()));
19      //diff_set = {3,9}
20
21      //積集合
22      std::set<int> inter_set;
23      std::set_intersection(a.begin(),a.end(),
24                             b.begin(),b.end(),
25                             std::inserter(inter_set,inter_set.end()));
26      //inter_set = {6}
27      return 0;
28  }

```

2.1.5 整数以外の集合

これまで整数を例に集合演算を紹介してきたが、整数以外を元にもつ集合も型を指定することによって作成が可能である。

```

1
2  //小数の集合
3  std::set<float> float_set = {0.1,0.01,0.002};
4
5  //文字の集合
6  // #include<string>が必要
7  std::set<char> char_set = {'I','S','C','T'};
8
9  //文字列の集合
10 std::set<std::string> string_set = {"Science","Tokyo"};

```

2.2 集合族

同様に、集合族を表すことも可能である。

```

1  //集合族
2  std::set<std::set<int>> s = {
3      {3,1},
4      {1,4},

```

```
5 {1,5}
6 };
```

2.3 std::set による集合表現の限界

前節で示したように、標準ライブラリに含まれるデータ構造と関数を使うだけでも集合に関するさまざまな演算を行うことができる。一見これらを示しただけでも「プログラミング言語で集合をどのように表現できるか」という問いに対する回答になっているかのようにも思える。

しかし、std::set には表現できない集合がある。std::set は内部で集合に含まれる要素を保持しているため、有限個（正確に言えば、コンピュータのメモリに収まる程度の個数）の要素しか扱うことができないのである。std::set は有限集合を、実際に値を保持することで管理することしかできないのである。有限集合しか表現できないということは、「補集合」も表現することができないということになる。

また、std::set はプログラミング言語特有の「型」というシステムに縛られている以上、異なる型の要素を含むような集合を表現することが難しい。「難しい」と言ったのは、不可能ではないからである。実際、以下のように記述すれば異なる型の要素を含むような集合を表現することが可能だからだ。

```
1 #include <iostream>
2 #include <set>
3 #include <variant>
4 #include <string>
5 using namespace std;
6 int main() {
7     set<std::variant<int, double, string>> mySet = {42, 3.14, "Tokyo"};
8
9     for (const auto& elem : mySet) {
10         std::visit([](const auto& arg) {
11             std::cout << arg << " ";
12         }, elem);
13     }
14     //42 3.14 Tokyo
15     return 0;
16 }
```

このような実装を可能にしているのが std::variant である。std::variant と std::visit を組み合わせることで、異なる型の要素を含む集合を作成することは可能である。しかしこの方法の利用には制約が多い。

まず、この方法では集合に入ってくる要素の型をあらかじめ列挙しておく必要がある。集合族を要素として扱うことも理論上は可能だが、「集合族の集合族」のようにネストが深くなっていくと、想定しておくべき要素の型が多くなってしまい、宣言が冗長であり直感的ではない。また、含み

得る型はコンパイル時に決定されてしまい、変更することができなくなる。新たな型を扱いたくなった場合は宣言自体を変更する必要があるのだ。

また、このままでは `std::set_intersection` などの標準ライブラリに含まれている関数を使うことができない。`std::variant` は強力だが、型システムによる制約を完全に回避できるわけではないのだ。

以降の章では、ここで挙げた 2 つの問題点を克服することも意識しつつ、自分なりのデータ構造を実装することを目指す。

3 関数による集合の表現

3.1 集合を関数で表現する

無限集合を扱えないという点を克服するため、「集合を関数で表現する」というアプローチをとることにする。`std::set` は集合に含まれる要素を列挙するという定義方法だった。集合の表し方には「外延的定義」と「内包的定義」があるが、`std::set` は外延的定義に対応していたことになる。それに対し、「集合を関数で表現する」というアプローチは内包的定義に対応していることになる。

集合を関数で表現するとは具体的にどういうことかと言うと、含まれる要素を実際に内部で保持するのではなく、「その値が集合に含まれているか？」を判定する関数だけを内部で保持しておくということだ。

以下にコードを示す。

```
1 #include <iostream>
2 #include <set>
3 #include <variant>
4 #include <string>
5 #include <algorithm>
6 #include <functional>
7
8 template<class T>
9 class SetByFunction{
10     std::function<bool(const T&)> pred;
11 public:
12     SetByFunction(std::function<bool(const T&)> pred)
13         : pred(pred) {}
14     bool contains(const T& x) const{
15         return pred(x);
16     }
17 };
18
19 int main(){
20     // 偶数全体の集合
```



```

21 SetByFunction<int> even_set([](int x){return x%2==0;});
22
23 std::cout << even_set.contains(2) << "\n";
24 //1
25
26 std::cout << even_set.contains(1) << "\n";
27 //0
28
29 std::cout << even_set.contains(999992) << "\n";
30 //1
31 return 0;
32 }

```

厳密には偶数全体の集合ではなく、int 型に収まる範囲での偶数全体の集合であることに注意しておきたい。しかし、範囲の問題に関しては無限の範囲を持つ型を使えばよいだけなので、あまり着目せずに以降の話を進める。

とにかく、このようにして内包的定義に沿って集合をプログラミング言語上で表すことができた。上の例では単に割ったあまりが 0 であるかを返す関数を持たせることで偶数全体の集合を表現したが、もっと複雑な関数を持たせることもできる。例えば素数全体の集合なども実現可能である。

3.2 集合演算と論理演算の対応

しかし、このままでは存在判定しか行うことができない。これだけでは集合を実用的に扱えるとは言えないため、集合演算についても実装を考える必要がある。ひとまず集合演算のうち

- 和集合
- 積集合
- 補集合
- 差集合

の 4 つの実装方法を考えてみよう。

結論から言えば、このような集合演算は論理演算と完全に対応している！ 和集合を最初の例として見ていってみよう。

3.2.1 和集合

集合 A, B に対して、その和集合 $A \cup B$ を考える。

$$A \cup B$$

これは「 A に含まれる、または B に含まれる要素全体からなる集合」である。

ここで、集合への所属判定を行う関数を

$$f_A(x) = \begin{cases} \text{true} & (x \in A) \\ \text{false} & (x \notin A) \end{cases}$$

と定義すると、和集合への所属判定は

$$x \in A \cup B \iff f_A(x) \vee f_B(x)$$

と表せる。

3.2.2 積集合

同様に集合 A, B に対して、その積集合 $A \cap B$ を考える。

$$x \in A \cap B \iff (x \in A) \wedge (x \in B)$$

したがって、所属判定関数では

$$f_{A \cap B}(x) = f_A(x) \wedge f_B(x)$$

となる。

3.2.3 補集合

集合 A に対して、その補集合 $\overline{A}$ を考える。

$$x \in \overline{A} \iff x \notin A$$

したがって、所属判定関数では

$$f_{\overline{A}}(x) = \neg f_A(x)$$

となる。

3.2.4 差集合

集合 A, B に対して、その差集合 $A - B$ を考える。

$$x \in A - B \iff (x \in A) \wedge (x \notin B)$$

したがって、所属判定関数では

$$f_{A-B}(x) = f_A(x) \wedge \neg f_B(x)$$

となる。

このように、4つの集合演算と論理演算が対応していることがわかった。実際に C++ で実装してみると、以下のようなになる。

```

1 #include <functional>
2 #include <iostream>
3
4 template<class T>
5 class SetByFunction{
6     std::function<bool(const T&)> pred;
7
8 public:
9     SetByFunction(std::function<bool(const T&)> pred)
10         : pred(pred){}
11
12     bool contains(const T& x) const{
13         return pred(x);
14     }
15
16     // 和集合
17     SetByFunction operator|(const SetByFunction& other) const{
18         auto f1 = pred;
19         auto f2 = other.pred;
20
21         return SetByFunction(
22             [f1, f2](const T& x){
23                 return f1(x) || f2(x);
24             }
25         );
26     }
27
28     // 積集合
29     SetByFunction operator&(const SetByFunction& other) const{
30         auto f1 = pred;
31         auto f2 = other.pred;
32         return SetByFunction(
33             [f1, f2](const T& x){
34                 return f1(x) && f2(x);
35             }
36         );
37     }
38
39     // 補集合
40     SetByFunction operator!() const{
41         auto f = pred;
42
43         return SetByFunction(
44             [f](const T& x){

```

```

45         return !f(x);
46     }
47     );
48 }
49
50 // 差集合
51 SetByFunction operator-(const SetByFunction& other) const{
52     auto f1 = pred;
53     auto f2 = other.pred;
54
55     return SetByFunction(
56         [f1, f2](const T& x){
57             return f1(x) && !f2(x);
58         }
59     );
60 }
61 };
62
63 int main(){
64
65     SetByFunction<int> even(
66         [](int x){ return x % 2 == 0; }
67     );
68
69     SetByFunction<int> multiple3(
70         [](int x){ return x % 3 == 0; }
71     );
72
73     auto A = even | multiple3;
74     auto B = even & multiple3;
75     auto C = !even;
76     auto D = even - multiple3;
77
78     std::cout << A.contains(6) << "\n"; // 1
79     std::cout << A.contains(4) << "\n"; // 1
80     std::cout << A.contains(5) << "\n"; // 0
81
82     std::cout << B.contains(6) << "\n"; // 1
83     std::cout << B.contains(4) << "\n"; // 0
84
85     std::cout << C.contains(5) << "\n"; // 1
86     std::cout << C.contains(4) << "\n"; // 0
87
88     std::cout << D.contains(4) << "\n"; // 1
89     std::cout << D.contains(6) << "\n"; // 0

```

90 }

演算子オーバーロードを使って実装し、2つのラムダ式を論理演算で合成する形となっている。演算子オーバーロードを使うことで、式の評価を C++ の標準機能に任せることができるのがポイントである。この実装であれば、

$$A \cup B \cup C$$

のようなシンプルな集合演算から、

$$(A \cup B) \cap (C \cup D)$$

のように括弧を使った演算にも対応できる。実際にコードを書いてみると、

```
1 //偶数 かつ 3の倍数 かつ 5の倍数
2 auto E = even & multiple3 & multiple5;
3 std::cout << E.contains(4) << "\n"; // 0
4 std::cout << E.contains(30) << "\n"; // 1
5
6 //(偶数 かつ 5の倍数)または(3の倍数 または 3を含む数)
7 auto F = (even & multiple5) | (multiple3 | contains3);
8
9 std::cout << F.contains(4) << "\n"; // 0
10 std::cout << F.contains(10) << "\n"; // 1
11 std::cout << F.contains(30) << "\n"; // 1
12 std::cout << F.contains(31) << "\n"; // 1
13 std::cout << F.contains(41) << "\n"; // 0
```

このように、集合を関数で表現することで、簡潔に集合および集合演算を表現することができた。

3.3 集合の一致判定

しかしここで問題になるのが、集合の一致判定である。集合を関数で表現するのはさまざまな長所があるが、集合の一致判定ができなくなるという問題がある。

その原因は、同じ集合を表す関数が一意に定まらないことである。実際に書いてみると、

```
1 //2で割ったあまりが0
2 auto A = SetByFunction<int>(<
3     [](int x){
4         return x%2==0;
5     }
6 );
7
8 //4で割ったあまりが0または2
9 auto B = SetByFunction<int>(<
10     [](int x){
11         return x%4==0 || x%4==2;
```

```
12     }  
13 );
```

この2つの集合が同じ集合であることは明らかだろう。しかし、「式が一致しているか」というだけで集合の一致判定を行うとこのようなケースで破綻してしまう。

したがって、「プログラムの構文」ではなく「プログラムが計算する関数の出力が一致しているか」を判定する必要がある。しかし計算可能性理論において、プログラムの計算する関数の入力と出力の振る舞い（意味的性質）に関する非自明な性質は一般に判定不能であることが知られている（ライス定理 [4]）。プログラムの等価性判定問題は停止性問題から帰着され、ライス定理からも導かれる代表的な決定不能問題の一つである。停止性問題を解くことが出来ないのは有名だが、停止性問題からさまざまな形の問題に帰着でき、プログラムの等価性判定も同様に解くことができないのだ。

また、「すべての要素が等しいかどうか」を判定することは、有限集合でない限り無限に検証する必要が発生してしまうため不可能である。

以上から、集合を関数で表現する方法で集合を実装すると集合の一致判定が出来ないということがわかる。プログラミングにおいて実用的に困らない範囲で表現することは難しくないが、数学的な意味で完全なものを作るのは難しいのである。

4 型の一般化

4.1 任意の型への対応

次に、任意の型を集合に含めることができるようにすることを考える。前章で実装した通り集合を関数で表現するという方針で進めよう。すると、任意の型を引数に取れる関数を作り、渡された型が違ったらその時点で `false` を返し、渡された型が正しかったら関数による判定を行う、という形にすれば良いだろう。

4.2 型消去による実装

実装の上では `std::any` を使って型を消去する。渡された関数の引数の型を推論してから初期化を行い、型の一致は `std::any_cast` で行った。

なお、出力内容が増えてきたため少し動作確認の出力の情報量を増やした。

```
1 #include <any>  
2 #include <functional>  
3 #include <iostream>  
4 #include <type_traits>  
5 #include <memory>  
6  
7 // ラムダ式を渡したときに引数の型を推論する  
8 template<class T>
```

```

9 struct callable_traits : callable_traits<decltype(&T::operator())> {};
10 //constを外す
11 template<class C, class R, class Arg>
12 struct callable_traits<R(C::*)(Arg) const> {
13     using arg_type = std::decay_t<Arg>;
14 };
15 //ポインタを外す
16 template<class C, class R, class Arg>
17 struct callable_traits<R(C::*)(Arg)> {
18     using arg_type = std::decay_t<Arg>;
19 };
20 //ラムダ式でない関数も渡せるように
21 template<class R, class Arg>
22 struct callable_traits<R(*)(Arg)> {
23     using arg_type = std::decay_t<Arg>;
24 };
25
26 class FunctionSetAnyType {
27     std::function<bool(const std::any&)> pred;
28
29     FunctionSetAnyType(std::function<bool(const std::any&)> p)
30         : pred(std::move(p)) {}
31
32 public:
33     template<class T>
34     FunctionSetAnyType(std::function<bool(const T&)> p) {
35         pred = [p](const std::any& x) -> bool {
36             if (const T* ptr = std::any_cast<T>(&x)) {
37                 return p(*ptr);
38             }
39             return false; // 型が違う要素はfalse
40         };
41     }
42
43     template<class F,
44             class = std::enable_if_t<!std::is_same_v<std::decay_t<F>,
45                                     FunctionSetAnyType>>>
46     FunctionSetAnyType(F f) : FunctionSetAnyType(std::function<bool(const
47         typename callable_traits<std::decay_t<F>>::arg_type&>(f)) {}
48
49     bool contains(const std::any& x) const {
50         return pred(x);
51     }
52
53     FunctionSetAnyType operator|(const FunctionSetAnyType& other) const {

```

```

52     auto f1 = pred, f2 = other.pred;
53     return FunctionSetAnyType([f1, f2](const std::any& x) { return f1(x)
        || f2(x); });
54 }
55 FunctionSetAnyType operator&(const FunctionSetAnyType& other) const {
56     auto f1 = pred, f2 = other.pred;
57     return FunctionSetAnyType([f1, f2](const std::any& x) { return f1(x)
        && f2(x); });
58 }
59 FunctionSetAnyType operator!() const {
60     auto f = pred;
61     return FunctionSetAnyType([f](const std::any& x) { return !f(x); });
62 }
63 FunctionSetAnyType operator-(const FunctionSetAnyType& other) const {
64     auto f1 = pred, f2 = other.pred;
65     return FunctionSetAnyType([f1, f2](const std::any& x) { return f1(x)
        && !f2(x); });
66 }
67 };
68
69 int main(){
70     FunctionSetAnyType even([](int x){ return x % 2 == 0; });
71     FunctionSetAnyType multiple3([](int x){ return x % 3 == 0; });
72     FunctionSetAnyType multiple5([](int x){ return x % 5 == 0; });
73
74     auto A = even | multiple3;
75     auto B = even & multiple3;
76     auto D = even - multiple3;
77
78     std::cout << "A(6)=" << A.contains(6) << " A(4)=" << A.contains(4) << "
        A(5)=" << A.contains(5) << "\n";
79     std::cout << "B(6)=" << B.contains(6) << " B(4)=" << B.contains(4) << "\n";
80     std::cout << "D(4)=" << D.contains(4) << " D(6)=" << D.contains(6) << "\n";
81     /*
82         A(6)=1 A(4)=1 A(5)=0
83         B(6)=1 B(4)=0
84         D(4)=1 D(6)=0
85     */
86     //任意の型への対応
87     std::cout << "----- 型が違う場合の挙動 -----\n";
88     std::cout << "even.contains(4) = " << even.contains(4) << "\n"; // int,
        OK -> 1
89     std::cout << "even.contains(4.0) = " << even.contains(4.0) << "\n"; //

```



```

double, 型不一致 -> 0
90 std::cout << "even.contains(std::string(\"x\")) = " << even.contains(std
    ::string("x")) << "\n"; // 0
91 /*
92 ----- 型が違う場合の挙動 -----
93 even.contains(4) = 1
94 even.contains(4.0) = 0
95 even.contains(std::string("x")) = 0
96 */
97
98 // 文字列の集合も同じ枠組みで扱える
99 FunctionSetAnyType startsWithA([](const std::string& s){ return !s.empty
    () && s[0] == 'A'; });
100 std::cout << "startsWithA(\"Apple\") = " << startsWithA.contains(std::
    string("Apple")) << "\n"; // 1
101 std::cout << "startsWithA(5) = " << startsWithA.contains(5) << "\n"; //
    型不一致 -> 0
102 /*
103 startsWithA("Apple") = 1
104 startsWithA(5) = 0
105 */
106
107 // 異なる型の集合同士でも同じ演算子で組み合わせられる
108 auto mixed = even | startsWithA;
109 std::cout << "mixed.contains(4) = " << mixed.contains(4) << "\n"; // 1 (
    even)
110 std::cout << "mixed.contains(\"Apple\") = " << mixed.contains(std::
    string("Apple")) << "\n"; // 1 (startsWithA)
111 std::cout << "mixed.contains(\"banana\") = " << mixed.contains(std::
    string("banana")) << "\n"; // 0
112 std::cout << "mixed.contains(3) = " << mixed.contains(3) << "\n"; // 0
113 /*
114 mixed.contains(4) = 1
115 mixed.contains("Apple") = 1
116 mixed.contains("banana") = 0
117 mixed.contains(3) = 0
118 */
119
120 return 0;
121 }

```

複数の型を含む集合に対応できていることがわかるだろう。もともと型システムに縛られた C++ で書いているのでやや記述量は多く、コードが冗長になっているが、主に行っているのは前述の通り「任意の型を引数に受け取る関数」を使った集合の実装である。

4.3 集合族への拡張

しかし、このままでは複数の型を含むことはできるが、集合族を持つことはできない。一見同じように実装を拡張すれば集合族も実装できるように思えるが、実際には原理的な問題がある。仮に実装をしたとしても実用性のあるものを作ることができないのだ。具体的には、そもそも存在判定ができないのである。なぜ存在判定が出来ないのかというと単純で、集合族の中に特定の集合が存在するかどうかを判定するには集合同士の等価判定ができる必要があり、これは前述したとおり無限集合の一致判定なので不可能だからだ。

5 有限集合との融合

前章で述べたように、関数で集合を表現する方法では集合族を実装することができないし、集合同士の一致判定を行うこともできない。これを部分的に解決することを考えよう。

集合同士の一致判定が出来ないのは、関数として表現された集合同士の比較が必要だからである。大まかに言えば、無限集合を扱うことによる困難さが原因である。関数で集合を表現する方法は無限集合の扱いには長けているが、有限集合の扱いは `std::set` の方が上回る形になっている。では、有限集合も無限集合も一括で扱えるようにしてしまえばよいのではないだろうか？

5.1 実装

ということで、これまで実装してきた関数による集合表現に、有限集合を扱う機能を追加することを考える。基本的には関数によって集合を表現し、その判定結果に対する例外として、有限個の要素を追加・削除できるようにする。

集合演算では関数の合成だけでなく有限集合も合成する必要があるため複雑になる。具体的には、集合を表す関数を `pred`、例外として含む要素を `in`、例外として含まない要素を `out` とすると、それぞれの集合演算は以下ようになる。

5.1.1 和集合

$$\begin{aligned} \text{in} &= (\text{in}_1 \setminus \text{out}_2) \cup (\text{in}_2 \setminus \text{out}_1) \\ \text{out} &= (\text{out}_1 \cap \neg \text{pred}_2) \cap (\text{out}_2 \cap \neg \text{pred}_1) \\ \text{pred} &= \text{pred}_1 \vee \text{pred}_2 \end{aligned}$$

5.1.2 積集合

$$\begin{aligned} \text{in} &= (\text{in}_1 \cap \text{pred}_2) \cup (\text{in}_2 \cap \text{pred}_1) \\ \text{out} &= (\text{out}_1 \setminus \text{in}_2) \cup (\text{out}_2 \setminus \text{in}_1) \\ \text{pred} &= \text{pred}_1 \wedge \text{pred}_2 \end{aligned}$$

5.1.3 差集合

$$\begin{aligned} \text{in} &= \text{in}_1 \cap \text{out}_2 \\ \text{out} &= \text{out}_1 \cup \text{in}_2 \\ \text{pred} &= \text{pred}_1 \wedge \neg \text{pred}_2 \end{aligned}$$

5.1.4 補集合

$$\begin{aligned} \text{in} &= \text{out} \\ \text{out} &= \text{in} \\ \text{pred} &= \neg \text{pred} \end{aligned}$$

これらをすべて実装したものが以下になる。std::any をそのまま使うと等式が判定できないので専用のラッパークラスで包んで使用している。有限集合を扱えるようにするためにコンストラクタを少し変更しつつ、デフォルトでは空の配列を指定しておくことでこれまでと同じ引数でも使えるようにしている。

なお、実装の簡略化のため集合演算についてはやや計算量が悪い実装となっている。

```
1 #include <any>
2 #include <functional>
3 #include <iostream>
4 #include <type_traits>
5 #include <memory>
6 #include <typeindex>
7 #include <vector>
8 // ラムダ式を渡したときに引数の型を推論する
9 template<class T>
10 struct callable_traits : callable_traits<decltype(&T::operator())> {};
11 //constを外す
12 template<class C, class R, class Arg>
13 struct callable_traits<R(C::*)(Arg) const> {
14     using arg_type = std::decay_t<Arg>;
15 };
16 //ポインタを外す
17 template<class C, class R, class Arg>
18 struct callable_traits<R(C::*)(Arg)> {
19     using arg_type = std::decay_t<Arg>;
20 };
21 //ラムダ式でない関数も渡せるように
22 template<class R, class Arg>
23 struct callable_traits<R(*) (Arg)> {
24     using arg_type = std::decay_t<Arg>;
```

```

25 };
26
27 //std::anyの代替 (比較を可能にするためのラッパークラス)
28 class AnyValue{
29     std::any value;
30     std::type_index type;
31     std::function<bool(const std::any&)> equal_;
32
33 public:
34     template<class T>
35     AnyValue(T x)
36         : value(std::move(x))
37         , type(typeid(T))
38     {
39         equal_ = [v = std::any_cast<T>(value)](const std::any& other)->bool{
40             if(const T* p = std::any_cast<T>(&other)){
41                 return *p == v;
42             }
43             return false;
44         };
45     }
46
47     const std::any& any() const{
48         return value;
49     }
50
51     template<class T>
52     const T* get_if() const{
53         return std::any_cast<T>(&value);
54     }
55
56     bool operator==(const AnyValue& rhs) const{
57         return type == rhs.type && equal_(rhs.value);
58     }
59
60     bool operator!=(const AnyValue& rhs) const{
61         return !(*this == rhs);
62     }
63 };
64
65 class FunctionSetAnyType {
66     std::function<bool(const std::any&)> pred;
67     std::vector<AnyValue>in_,out_;
68
69     FunctionSetAnyType(std::function<bool(const std::any&)> p,

```

```

70     std::vector<AnyValue>f_in = {},
71     std::vector<AnyValue>f_out = {})
72     : pred(std::move(p)),in_(std::move(f_in)),out_(std::move(f_out)) {}
73
74 public:
75     template<class T>
76     FunctionSetAnyType(std::function<bool(const T&)> p,std::vector<AnyValue>
77         f_in = {},std::vector<AnyValue>f_out = {}) {
78         pred = [p](const std::any& x) -> bool {
79             if (const T* ptr = std::any_cast<T>(&x)) {
80                 return p(*ptr);
81             }
82             return false; // 型が違う要素はfalse
83         };
84         in_ = f_in;
85         out_ = f_out;
86     }
87
88     template<class F,
89         class = std::enable_if_t<!std::is_same_v<std::decay_t<F>,
90             FunctionSetAnyType>>>
91     FunctionSetAnyType(F f,std::vector<AnyValue>f_in = {},std::vector<
92         AnyValue>f_out = {}) : FunctionSetAnyType(std::function<bool(const
93             typename callable_traits<std::decay_t<F>>::arg_type&>(f),f_in,f_out
94         ) {}
95
96     bool contains(const AnyValue& x) const{
97         for(const auto& v:out_)
98             if(v==x) return false;
99
100         for(const auto& v:in_)
101             if(v==x) return true;
102
103         return pred(x.any());
104     }
105
106     FunctionSetAnyType operator|(const FunctionSetAnyType& other) const {
107         auto f1 = pred, f2 = other.pred;
108
109         auto new_pred = [f1, f2](const std::any& x) {
110             return f1(x) || f2(x);
111         };
112
113         std::vector<AnyValue> cand = in_;
114         auto add = [&](const std::vector<AnyValue>& v){

```

```

110         for(const auto& x : v)
111             if(std::find(cand.begin(), cand.end(), x) == cand.end())
112                 cand.push_back(x);
113     };
114     add(out_);
115     add(other.in_);
116     add(other.out_);
117
118     std::vector<AnyValue> new_in, new_out;
119
120     for(const auto& x : cand){
121         bool ans = contains(x) || other.contains(x);
122         bool p = new_pred(x.any());
123
124         if(ans != p){
125             if(ans) new_in.push_back(x);
126             else new_out.push_back(x);
127         }
128     }
129
130     return FunctionSetAnyType(new_pred, new_in, new_out);
131 }
132
133 FunctionSetAnyType operator&(const FunctionSetAnyType& other) const {
134     auto f1 = pred, f2 = other.pred;
135
136     auto new_pred = [f1, f2](const std::any& x) {
137         return f1(x) && f2(x);
138     };
139
140     std::vector<AnyValue> cand = in_;
141     auto add = [&](const std::vector<AnyValue>& v){
142         for(const auto& x : v)
143             if(std::find(cand.begin(), cand.end(), x) == cand.end())
144                 cand.push_back(x);
145     };
146     add(out_);
147     add(other.in_);
148     add(other.out_);
149
150     std::vector<AnyValue> new_in, new_out;
151
152     for(const auto& x : cand){
153         bool ans = contains(x) && other.contains(x);
154         bool p = new_pred(x.any());

```

```

155
156         if(ans != p){
157             if(ans) new_in.push_back(x);
158             else new_out.push_back(x);
159         }
160     }
161
162     return FunctionSetAnyType(new_pred, new_in, new_out);
163 }
164 FunctionSetAnyType operator!() const {
165     auto f = pred;
166     return FunctionSetAnyType([f](const std::any& x) { return !f(x); },
167                                out_, in_);
168 }
169 FunctionSetAnyType operator-(const FunctionSetAnyType& other) const {
170     auto f1 = pred, f2 = other.pred;
171     std::vector<AnyValue> new_in;
172     for(AnyValue x:in_){
173         auto it = std::find(other.out_.begin(), other.out_.end(), x);
174         if(it != other.out_.end()){
175             new_in.push_back(x);
176         }
177     }
178     std::vector<AnyValue> new_out = out_;
179     for(AnyValue x:other.in_){
180         auto it = std::find(new_out.begin(), new_out.end(), x);
181         if(it == new_out.end()){
182             new_out.push_back(x);
183         }
184     }
185     return FunctionSetAnyType([f1, f2](const std::any& x) { return f1(x)
186                                     && !f2(x); }, new_in, new_out);
187 }
188 };
189
190 int main(){
191     FunctionSetAnyType even([](int x){ return x % 2 == 0; });
192     FunctionSetAnyType multiple3([](int x){ return x % 3 == 0; });
193     FunctionSetAnyType multiple5([](int x){ return x % 5 == 0; });
194     std::cout << "\n===== 有限集合のテスト =====\n";
195
196     // 偶数集合だが 3 は例外として追加、2 は例外として削除
197     FunctionSetAnyType specialEven(
198         [](int x){ return x % 2 == 0; },
199         {3},

```

```

196     {2}
197 );
198
199 std::cout << "specialEven.contains(2) = "
200           << specialEven.contains(2)
201           << " (expected 0)\n";
202
203 std::cout << "specialEven.contains(3) = "
204           << specialEven.contains(3)
205           << " (expected 1)\n";
206
207 std::cout << "specialEven.contains(4) = "
208           << specialEven.contains(4)
209           << " (expected 1)\n";
210
211 std::cout << "specialEven.contains(5) = "
212           << specialEven.contains(5)
213           << " (expected 0)\n";
214
215 std::cout << "\n===== 和集合 =====\n";
216
217 auto U = specialEven | multiple3;
218
219 std::cout << "U.contains(2) = "
220           << U.contains(2)
221           << " (expected 0)\n"; // out優先
222
223 std::cout << "U.contains(3) = "
224           << U.contains(3)
225           << " (expected 1)\n";
226
227 std::cout << "U.contains(9) = "
228           << U.contains(9)
229           << " (expected 1)\n";
230
231 std::cout << "U.contains(4) = "
232           << U.contains(4)
233           << " (expected 1)\n";
234
235
236 std::cout << "\n===== 積集合 =====\n";
237
238 auto I = specialEven & multiple3;
239
240 std::cout << "I.contains(3) = "

```



```

241         << I.contains(3)
242         << " (expected 1)\n"; // inに残るはず
243
244 std::cout << "I.contains(6) = "
245         << I.contains(6)
246         << " (expected 1)\n";
247
248 std::cout << "I.contains(4) = "
249         << I.contains(4)
250         << " (expected 0)\n";
251
252
253 std::cout << "\n===== 差集合 =====\n";
254
255 auto S = specialEven - multiple3;
256
257 std::cout << "S.contains(3) = "
258         << S.contains(3)
259         << " (expected 0)\n";
260
261 std::cout << "S.contains(4) = "
262         << S.contains(4)
263         << " (expected 1)\n";
264
265 std::cout << "S.contains(6) = "
266         << S.contains(6)
267         << " (expected 0)\n";
268
269
270 std::cout << "\n===== 補集合 =====\n";
271
272 auto C = !specialEven;
273
274 std::cout << "C.contains(2) = "
275         << C.contains(2)
276         << " (expected 1)\n";
277
278 std::cout << "C.contains(3) = "
279         << C.contains(3)
280         << " (expected 0)\n";
281
282 std::cout << "C.contains(4) = "
283         << C.contains(4)
284         << " (expected 0)\n";
285

```

```

286 std::cout << "C.contains(5) = "
287         << C.contains(5)
288         << " (expected 1)\n";
289
290     return 0;
291 }

```

実際に動かして想定通りの出力が得られることを確認できた。これにて当初実装したかったものを全部実装することができた！！

6 DSL による集合記法

Web 上で動かすために、集合の形で書いて先ほどのデータ構造を利用することができるようにする。

6.1 設計方針

全ての機能に対応させることは難しく、特に関数で表現する方法では様々な書き方ができてしまうため、ひとまず限定的な機能を実装した。具体的には、集合の初期化では有限集合と、 $x\%3=0$ のように割ったあまりを指定する関数のみに対応している。

6.2 記法の例

```

1 a = {x%2==0}
2 b = {4,5}
3 c = a | b
4 d = a & b
5 e = !a
6 f = (a | b) & !d
7 contains(a,4)
8 contains(b,7)

```

6.3 実装

構文解析をする都合上、コードはかなり長くなっている。この実装を読む必要はないだろう。

実際に <https://blueberry1001.github.io/#/wasmtest> にアクセスすることで動かすことができる。是非一度試してみてほしい。

```

1 using It = std::string::const_iterator;
2

```

```

3 void skip(It& it, It ed){
4     while(it≠ed && isspace(*it)) ++it;
5 }
6
7 bool consume(It& it, It ed, char c){
8     skip(it,ed);
9     if(it≠ed && *it=c){
10         ++it;
11         return true;
12     }
13     return false;
14 }
15
16 std::string parseIdentifier(It& it, It ed){
17     skip(it,ed);
18
19     std::string s;
20     while(it≠ed && (isalnum(*it)||*it=='_')){
21         s.push_back(*it++);
22     }
23
24     if(s.empty()) throw std::runtime_error("identifier expected");
25     return s;
26 }
27
28 bool isPredicate(It it, It ed){
29     int dep=0;
30     while(it≠ed){
31         if(*it=='{') dep++;
32         else if(*it=='}'){
33             dep--;
34             if(dep==0) break;
35         }
36         else if(dep==1 && *it=='%')
37             return true;
38         ++it;
39     }
40     return false;
41 }
42
43 FunctionSetAnyType parseFiniteSet(It& it, It ed){
44     consume(it,ed,'{');
45
46     std::vector<AnyValue> in;
47

```

```

48 while(true){
49     skip(it,ed);
50
51     std::string token;
52
53     while(it≠ed && *it≠',' && *it≠'}')
54         token.push_back(*it++);
55
56     in.emplace_back(std::stoi(token));
57
58     skip(it,ed);
59
60     if(consume(it,ed,','))
61         break;
62
63     consume(it,ed,');
64 }
65
66 return FunctionSetAnyType(
67     [](int){return false;},
68     in,
69     {}
70 );
71 }
72
73
74 FunctionSetAnyType parsePredicate(It& it, It ed){
75     consume(it,ed, '{');
76
77     skip(it,ed);
78
79     // x
80     if(!consume(it,ed, 'x'))
81         throw std::runtime_error("expected x");
82
83     skip(it,ed);
84
85     // %
86     if(!consume(it,ed, '%'))
87         throw std::runtime_error("expected %");
88
89     skip(it,ed);
90
91     // divisor
92     std::string s;

```

```

93     while(it≠ed && isdigit(*it)){
94         s.push_back(*it++);
95     }
96     if(s.empty())
97         throw std::runtime_error("number expected");
98
99     int mod = std::stoi(s);
100
101     skip(it,ed);
102
103     // =
104     if(!(consume(it,ed,'=') && consume(it,ed,'=')))
105         throw std::runtime_error("expected ==");
106
107     skip(it,ed);
108
109     s.clear();
110     while(it≠ed && isdigit(*it)){
111         s.push_back(*it++);
112     }
113     if(s.empty())
114         throw std::runtime_error("number expected");
115
116     int rem = std::stoi(s);
117
118     skip(it,ed);
119
120     if(!consume(it,ed,'}'))
121         throw std::runtime_error("expected }");
122
123     return FunctionSetAnyType(
124         [=](int x){
125             return x % mod == rem;
126         }
127     );
128 }
129
130 FunctionSetAnyType parseExpr(It&,It);
131 std::unordered_map<std::string,FunctionSetAnyType> variables;
132
133 FunctionSetAnyType parseFactor(It& it, It ed){
134
135     skip(it,ed);
136
137     if(consume(it,ed,'!'))

```

```

138     return !parseFactor(it,ed);
139
140     if(consume(it,ed,'(')){
141         auto t=parseExpr(it,ed);
142         consume(it,ed,')');
143         return t;
144     }
145
146     if(*it=='{'){
147         if(isPredicate(it,ed))
148             return parsePredicate(it,ed);
149         else
150             return parseFiniteSet(it,ed);
151     }
152
153     auto name=parseIdentifier(it,ed);
154     return variables.at(name);
155 }
156 FunctionSetAnyType parseTerm(It& it, It ed){
157
158     auto lhs=parseFactor(it,ed);
159
160     while(true){
161
162         skip(it,ed);
163
164         if(!consume(it,ed,'&'))
165             break;
166
167         lhs = lhs & parseFactor(it,ed);
168     }
169
170     return lhs;
171 }
172 FunctionSetAnyType parseExpr(It& it, It ed){
173
174     auto lhs=parseTerm(it,ed);
175
176     while(true){
177
178         skip(it,ed);
179
180         if(!consume(it,ed,'|'))
181             break;
182

```

```

183     lhs = lhs | parseTerm(it,ed);
184 }
185
186     return lhs;
187 }
188 void execute(std::string line){
189
190     It it=line.begin();
191
192     skip(it,line.end());
193
194     // contains(a,5)
195     if(line.find("contains") == 0){
196
197         consume(it,line.end(),'c');
198
199         std::string dummy;
200         while(it≠line.end() && *it≠'(')
201             ++it;
202
203         consume(it,line.end(),'(');
204
205         auto name=parseIdentifier(it,line.end());
206
207         consume(it,line.end()),',');
208
209         std::string value;
210
211         while(it≠line.end() && *it≠')'){
212             value.push_back(*it++);
213         }
214
215         consume(it,line.end()),')');
216
217         int x=std::stoi(value);
218
219         // std::cout
220         // << name << ".contains("
221         // << x << ") = "
222         // << variables.at(name).contains(x)
223         // << "\n";
224
225         return;
226     }
227

```

```
228
229 // 通常の代入
230 auto name=parseIdentifier(it,line.end());
231
232 consume(it,line.end(),'=');
233
234 variables[name]=parseExpr(it,line.end());
235 }
```


7 まとめ

7.1 本ライブラリの特徴

`std::set` と比較した本ライブラリの特徴は、無限集合を扱えることと、任意の型に対応していることである。集合を関数として表現するというアプローチにより、有限集合だけでなく条件によって定義される無限集合も扱うことを可能にした。また、関数による表現で生じる課題についても、有限集合との組み合わせによって対応した。

7.2 今後の課題

本ライブラリは機能面を重視して設計しており、速度面については十分な最適化を行っていない。そのため、計算量の改善や内部表現の工夫によって、さらなる高速化が期待できる。また、Web上で公開しているデモについても、現在は対応している構文や機能が限定的であり、今後さらに拡張する余地があるだろう。

8 アピール文

`std::set` では扱うことができない「無限集合」「任意の型の集合」を扱えるように独自のデータ構造を設計・実装した。「集合を関数で表現する」というアプローチによって無限集合をスマートに表すことができたが、制作過程で関数設計をしていくうちに、関数で表現することによるデメリット（例えば集合の同一性が判定できないなど）にも気づくことができた。本レポートは `std::set` から出発して、独自データ構造の設計・実装をし、最後には Web 上で動くデモまで作成した過程をまとめたものとなっている。情報理工学基礎の授業で学んだ集合論をプログラミング言語上に実装するという課題に対して、単なる集合演算の実装に留まらず、データ構造の設計や表現方法について考察することで、発展的な内容を含むレポートとなった。

なお、<https://blueberry1001.github.io/#/wasmtest> にアクセスすることでデモを動かすことができる。

参考文献

- [1] `cppreference.com`, *C++ reference*, <https://en.cppreference.com/>
- [2] `cpprefjp`, *cpprefjp - C++ 日本語リファレンス*, <https://cpprefjp.github.io/>
- [3] Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013.
<https://www.stroustrup.com/4th.html>

- [4] H. G. Rice, *Classes of Recursively Enumerable Sets and Their Decision Problems*, Trans. Amer. Math. Soc., Vol. 74, No. 2, pp. 358–366, 1953. <https://www.semanticscholar.org/paper/Classes-of-recursively-enumerable-sets-and-their-Rice/664a7d3c60b753a34f1601a7378ca952ea92e9a8>
- [5] University of Rochester, *CSC 173: Computability and Undecidability*, https://www.cs.rochester.edu/u/nelson/courses/csc_173/computability/undecidable.html